

Introduction to Python for Business Statistics

A 6-Week Intensive Course

This textbook accompanies a 6-week intensive course designed for graduate certificate students in Business Intelligence and Data Analytics. Python concepts are introduced through business statistics — students build both skills simultaneously, each reinforcing the other.

© 2026 Patrick Dolinger — Licensed under Creative Commons Attribution 4.0 International (CC BY 4.0)

Week	Python	Statistics
1	Variables, types, arithmetic	Data types, measurement scales
2	Lists, NumPy arrays	Central tendency, samples vs populations
3	Conditionals, loops, comprehensions	Frequency distributions, outlier detection
4	Functions, error handling, Pandas	Variance, std deviation, z-scores, IQR
5	Data cleaning, Matplotlib, Seaborn	Distributions, correlation, visualisation
6	groupby, pivot tables, EDA workflow	Grouped analysis, coefficient of variation

Chapter 1 — Data, Variables, and Types

1.1 What Is Data?

Data is recorded information. In a business context, data is collected whenever something is measured, counted, or categorised — a sale is recorded, a customer leaves a rating, a shipment is logged.

Before writing a single line of Python, it helps to think about what kind of data you are working with. Not all data is the same, and the type of data determines what you can do with it.

1.2 Measurement Scales

Every variable in a dataset belongs to one of four **measurement scales**. This concept comes from statistics, but it has direct practical consequences in Python — it determines which operations and analyses are valid.

Scale	Description	Example
Nominal	Categories with no meaningful order	Product name, sales region, payment method
Ordinal	Categories with a meaningful order, but unequal gaps	Customer satisfaction (1–5 stars), performance rating
Interval	Ordered with equal gaps, but no true zero	Year, temperature in °C
Ratio	Ordered with equal gaps and a true zero	Revenue, units sold, profit margin

The key distinction between interval and ratio is the **true zero**. A true zero means the value zero represents a genuine absence of the thing being measured. Revenue of \$0 means no revenue was earned — that is a true zero. The year 0 does not mean "no time" — so year is interval, not ratio.

***Why it matters:** You can calculate the mean of a ratio variable (average revenue), but calculating the mean of a nominal variable (average product name) is meaningless. Calculating the mean of an ordinal variable (average star rating) is technically possible but*

statistically questionable, since the gaps between 1 and 2 stars may not equal the gap between 4 and 5 stars.

1.3 Python Variables

A **variable** is a named container that stores a value. You create one using the assignment operator `=`.

```
sale_amount = 349.95
```

The variable name goes on the left, the value on the right. From this point forward, anywhere you use `sale_amount` in your code, Python substitutes the value 349.95.

Naming Rules

Variable names in Python must follow these rules:

- Start with a letter or underscore, never a number
- Contain only letters, numbers, and underscores
- Cannot be a Python reserved word (such as `if`, `for`, `True`)

By convention, Python variable names use **snake_case** — all lowercase with underscores between words. This is the standard in data work.

```
# Good names
sale_amount = 349.95
product_name = "Laptop Stand"
is_returned = False

# Avoid
SaleAmount = 349.95      # not snake_case
sa = 349.95              # too short, not descriptive
```

1.4 Core Data Types

Python has four data types you will use in almost every analysis.

Integer — `int`

Whole numbers. Used for counts, IDs, and quantities that cannot be fractional.

```
units_sold = 150
transaction_id = 1042
fiscal_year = 2023
```

Float — `float`

Decimal numbers. Used for prices, rates, percentages, and most financial values.

```
sale_amount = 349.95
tax_rate = 0.13
margin = 0.4275
```

String — str

Text data. Used for names, categories, labels, and any value that is not a number. Always enclosed in quotes.

```
product_name = "Laptop Stand"
sales_region = "Northeast"
payment_method = "Credit Card"
```

Boolean — bool

True or false values. Used for flags and binary conditions.

```
is_returned = False
is_online_sale = True
is_discounted = False
```

Checking a Variable's Type

Use the built-in `type()` function to confirm what type a variable is.

```
print(type(sale_amount))    # <class 'float'>
print(type(units_sold))    # <class 'int'>
print(type(product_name))  # <class 'str'>
print(type(is_returned))   # <class 'bool'>
```

Connecting Types to Measurement Scales

Python types and statistical measurement scales are related but not identical. The same Python type can represent different scales depending on context.

Scale	Typical Python Type	Notes
Nominal	str, sometimes int	An int used as a postal code or product ID is nominal
Ordinal	int or str	A rating stored as int is still ordinal — the gaps are unequal
Interval	int or float	Year is int, but is interval scale
Ratio	int or float	Revenue, units sold, quantities

The type tells Python how to store the value. The scale tells you what statistical operations are valid.

1.5 Arithmetic Operations

Python supports standard arithmetic. The results are used constantly in business analysis — calculating totals, averages, growth rates, and proportions.

Operator	Operation	Example	Result
+	Addition	10 + 3	13
-	Subtraction	10 - 3	7
*	Multiplication	10 * 3	30
/	Division	10 / 3	3.333...
//	Floor division	10 // 3	3
%	Remainder	10 % 3	1
**	Exponentiation	10 ** 3	1000

Note that `/` always returns a `float`, even if the result is a whole number. `//` returns an `int` by discarding the decimal.

```
q1_revenue = 12400.00
q2_revenue = 15800.00
q3_revenue = 13950.00
q4_revenue = 19200.00

annual_revenue = q1_revenue + q2_revenue + q3_revenue + q4_revenue
print(annual_revenue)    # 61350.0
```

1.6 The Mean

The **mean** is the most commonly used measure of central tendency. It is the sum of all values divided by the count of values.

$$\bar{x} = \frac{\sum x}{n}$$

Where $\bar{x}$ is the mean, $\sum x$ is the sum of all values, and n is the number of values.

```
average_quarterly_revenue = annual_revenue / 4
print(average_quarterly_revenue)    # 15337.5
```

The mean is appropriate for **interval** and **ratio** scale variables. Applying it to nominal or ordinal data produces a number, but that number has no meaningful interpretation.

1.7 String Operations

Strings represent categorical and text data. Python provides several tools for working with them cleanly.

Common String Methods

```
product = "laptop stand"

print(product.upper())    # LAPTOP STAND
print(product.title())    # Laptop Stand
print(product.lower())    # laptop stand
print(len(product))       # 12
```

Concatenation

Strings can be joined using +.

```
product = "Laptop Stand"
region = "Northeast"
label = product + " – " + region
print(label)    # Laptop Stand – Northeast
```

f-Strings

f-strings are the cleanest way to embed variable values inside a text output. Prefix the string with `f` and place variable names inside `{}`.

```
product = "Laptop Stand"
units = 47
region = "Northeast"

print(f"{product} sold {units} units in {region}.")
# Laptop Stand sold 47 units in Northeast.
```

f-strings also support formatting specifiers. The most useful for business data is `:.2f`, which rounds a float to two decimal places.

```
revenue = 8420.5
print(f"Revenue: ${revenue:.2f}")    # Revenue: $8420.50
```

1.8 Chapter Summary

Concept	Key Point
Measurement scales	Nominal, ordinal, interval, ratio — determines valid operations
Variables	Named containers assigned with =, use snake_case
int	Whole numbers — counts, IDs, years
float	Decimal numbers — prices, rates, percentages
str	Text — names, categories, labels
bool	True/False — flags and binary conditions
type()	Returns the data type of any variable
Mean	Sum divided by count — valid for interval and ratio only
f-strings	Clean way to embed variables in printed output

Review Questions

1. A dataset contains a column called `customer_id` stored as integers (e.g., 10042, 10043). What measurement scale is this variable? Can you calculate its mean? Why or why not?
2. A sales rep's performance is rated as Bronze, Silver, or Gold. What measurement scale is this? What Python type would you use to store it?
3. What is the difference between `/` and `//` in Python? Give an example where the distinction matters.
4. Write a Python statement that calculates the percentage change in revenue from \$48,000 to \$54,600.
5. A colleague argues that you can average customer star ratings (1–5) to get a "mean satisfaction score." What is the statistical argument against this? When might you do it anyway?

Chapter 2 — Lists, Arrays, and Central Tendency

2.1 From Single Values to Collections

In Week 1 we stored one value per variable. Real data rarely works that way. A sales dataset has hundreds of transactions. A monthly report has twelve figures. An employee file has one row per person.

Python provides several ways to store collections of values. This chapter covers the two you will use most in data work: **lists** and **NumPy arrays**.

2.2 Python Lists

A **list** is an ordered collection of values stored in a single variable. Values are enclosed in square brackets and separated by commas.

```
monthly_sales = [14200, 13850, 16400, 15900, 17200, 18100]
products = ["Laptop Stand", "Monitor Arm", "Keyboard Tray", "Cable Manager"]
```

Lists can hold any data type — integers, floats, strings, or a mix. In data work, a single list typically holds one type consistently, mirroring a column in a spreadsheet.

Useful List Functions

Function	What It Does	Example
<code>len()</code>	Count of items	<code>len(monthly_sales) → 6</code>
<code>sum()</code>	Sum of numeric values	<code>sum(monthly_sales) → 95650</code>
<code>min()</code>	Smallest value	<code>min(monthly_sales) → 13850</code>
<code>max()</code>	Largest value	<code>max(monthly_sales) → 18100</code>
<code>sorted()</code>	Returns a sorted copy	<code>sorted(monthly_sales)</code>

2.3 Indexing

Every item in a list has a position called an **index**. Python indices start at **0**.


```
monthly_sales = [14200, 13850, 16400, 15900, 17200, 18100]
index:          0      1      2      3      4      5
```

Access an item by placing its index in square brackets.

```
monthly_sales[0]    # 14200 – January
monthly_sales[2]    # 16400 – March
monthly_sales[5]    # 18100 – June
```

Negative Indexing

Negative indices count from the end of the list. This is useful when you want the last item without knowing the list's length.

```
monthly_sales[-1]   # 18100 – last item (June)
monthly_sales[-2]   # 17200 – second to last (May)
```

2.4 Slicing

A **slice** extracts a portion of a list. The syntax is `list[start:stop]`, where the stop index is **not included**.

```
monthly_sales[0:3]  # [14200, 13850, 16400] – January to March
monthly_sales[3:6]  # [15900, 17200, 18100] – April to June
```

Omitting either boundary defaults to the start or end of the list.

```
monthly_sales[:3]   # first three items – same as [0:3]
monthly_sales[3:]   # from index 3 to the end
```

This maps naturally to business data. Extracting a quarter, a fiscal half-year, or the most recent period from a list is a slicing operation.

2.5 NumPy Arrays

The Python list is general-purpose. For numerical data, **NumPy** (Numerical Python) provides a more powerful structure: the **array**.

NumPy arrays are faster than lists for numerical work and support operations that would otherwise require loops. They are the foundation of most data analysis in Python.

Import NumPy using the standard alias `np`.

```
import numpy as np

sales_array = np.array([14200, 13850, 16400, 15900, 17200, 18100])
```

Element-Wise Arithmetic

The most immediate advantage of NumPy arrays is **element-wise arithmetic** — an operation applied to the whole array at once, without needing to loop through each item.

```
# Apply a 10% revenue increase to every month
adjusted = sales_array * 1.10
# [15620., 15235., 18040., 17490., 18920., 19910.]

# Subtract a fixed monthly overhead
net = sales_array - 2000
# [12200, 11850, 14400, 13900, 15200, 16100]
```

With a plain Python list, each of these would require a loop. With a NumPy array, one line handles it. This is a recurring pattern in data work.

Lists vs Arrays

	Python List	NumPy Array
Holds mixed types	Yes	No — all elements must be the same type
Element-wise math	No — requires a loop	Yes — direct operations
Speed on large data	Slow	Fast
Statistical functions	Limited	Extensive (np.mean, np.std, etc.)

Use lists for general-purpose storage. Use NumPy arrays whenever you are doing numerical computation.

2.6 Samples and Populations

Before computing any statistics, it is worth being clear about what your data represents.

A **population** is the complete set of all cases you are interested in. A **sample** is a subset of that population — the data you actually have.

	Population	Sample
Definition	All cases of interest	A subset of the population
Size notation	N	n
Mean notation	μ (μ)	$\bar{x}$ ($\bar{x}$)

In most real business scenarios, you are working with a **sample**. Six months of sales data is a sample from the company's full sales history. A survey of 500 customers is a sample from all customers.

This distinction matters because some statistical formulas differ slightly between sample and population calculations. You will see this directly in Week 4 when we cover variance and standard deviation.

2.7 Measures of Central Tendency

A **measure of central tendency** summarises a dataset with a single value representing its centre. There are three standard measures: the mean, the median, and the mode.

The Mean

The mean is the sum of all values divided by the count.

$$\bar{x} = \frac{\sum x}{n}$$

```
mean = np.mean(sales_array)    # 15941.67
```

The mean uses every value in the dataset. This makes it precise but sensitive — a single extreme value can shift it significantly.

The Median

The median is the middle value of a sorted dataset.

- If n is **odd**: the median is the middle value
- If n is **even**: the median is the average of the two middle values

```
median = np.median(sales_array)    # 15950.0
```

With six values sorted as [13850, 14200, 15900, 16400, 17200, 18100], the median is the average of the 3rd and 4th values: $(15900 + 16400) / 2 = 16150$.

The median ignores the actual magnitude of values above and below it. This makes it **resistant to outliers**.

The Mode

The mode is the value that occurs most frequently.

```
from scipy import stats
ratings = np.array([3, 4, 4, 5, 3, 4, 2, 5, 4, 3])
result = stats.mode(ratings, keepdims=True)
# Mode: 4, appears 4 times
```

The mode is the only measure of central tendency valid for **nominal** data. You can find the most common product category, the most frequent payment method, or the most common sales region. The mean and median of those categories would be meaningless.

2.8 Choosing the Right Measure

Situation	Best Measure	Reason
Symmetric data, no outliers	Mean	Uses all data, most precise
Skewed data or outliers present	Median	Not distorted by extreme values
Nominal or categorical data	Mode	Only valid option
Reporting typical salary or house price	Median	Outliers (executives, luxury properties) distort the mean

The Outlier Problem

Consider two sets of monthly sales figures:

```
normal = [14200, 13850, 16400, 15900, 17200, 18100]
outlier = [14200, 13850, 16400, 15900, 17200, 95000]
```

	Mean	Median
Normal	\$15,941	\$15,950
With outlier	\$28,758	\$16,150

The outlier more than doubles the mean. The median moves by only \$200. When one month included an exceptional bulk order, the median is a far more representative figure for planning and forecasting purposes.

This is why news reports on income and housing prices typically use **median** rather than mean. A small number of very high values would otherwise give a misleading picture of what is typical.

2.9 Chapter Summary

Concept	Key Point
---------	-----------

List	Ordered collection, indexed from 0, supports mixed types
Indexing	<code>list[0]</code> = first, <code>list[-1]</code> = last
Slicing	<code>list[start:stop]</code> — stop is excluded
NumPy array	Numerical collection with element-wise arithmetic and statistical functions
Population vs sample	Population = all cases (N , μ); Sample = subset (n , $\bar{x}$)
Mean	Sum / count — sensitive to outliers
Median	Middle value — resistant to outliers
Mode	Most frequent value — valid for nominal data

Review Questions

1. What is the difference between `monthly_sales[3]` and `monthly_sales[3:6]`? What does each return?
2. A dataset contains annual revenues for 500 companies, including 5 companies worth over \$10 billion. Would you report the mean or median revenue as the "typical" company revenue? Justify your answer.
3. A list contains 9 values. After sorting, the values are: [12, 15, 18, 21, 24, 27, 30, 33, 36]. Calculate the median by hand, then verify using `np.median()`.
4. Why can you not calculate a meaningful mean for the variable `sales_region` even if you encoded the regions as numbers (1 = Northeast, 2 = West, 3 = South)?
5. You have a NumPy array of daily sales figures for a year (365 values). Write a single line of code that increases every value by 5% and stores the result in a new variable called `projected_sales`.

Chapter 3 — Conditionals, Loops, and Frequency Distributions

3.1 Making Decisions with Conditionals

Most business logic involves decisions: flag a transaction above a threshold, classify a customer by spending tier, mark a month as above or below target. In Python, these decisions are handled by **conditional statements**.

```
if condition:
    # runs when condition is True
elif other_condition:
    # runs when other_condition is True
else:
    # runs when no condition above is True
```

Indentation is not optional in Python — it defines what code belongs inside each block. Consistent use of four spaces is the standard.

Comparison Operators

Operator	Meaning	Example
==	Equal to	sales == target
!=	Not equal to	region != "West"
>	Greater than	sales > 10000
<	Less than	sales < 10000
>=	Greater than or equal	rating >= 4
<=	Less than or equal	discount <= 0.20

Note the difference between = (assignment, stores a value) and == (comparison, tests equality). Confusing the two is one of the most common beginner errors.

Logical Operators

Multiple conditions can be combined using and, or, and not.

```
if sales > 10000 and region == "Northeast":  
    print("High-performing Northeast sale")  
  
if sales < 1000 or is_returned == True:  
    print("Flag for review")
```

and requires both conditions to be true. or requires at least one to be true.

An Example

```
monthly_sales = 17200  
target        = 15000  
  
if monthly_sales >= target * 1.10:  
    print("Exceeded target by 10% or more")  
elif monthly_sales >= target:  
    print("Met target")  
else:  
    print("Below target")
```

Python evaluates each condition in order, top to bottom. As soon as one condition is true, it runs that block and skips the rest. Order matters — place the most specific conditions first.

3.2 For Loops

A **for loop** repeats a block of code once for each item in a collection. This is how you process every row in a dataset, every month in a time series, or every item in a list.

```
for item in collection:  
    # code that runs once per item
```

Iterating with Index — `range()` **and** `enumerate()`

Sometimes you need both the item and its position. Two approaches:

```
# Using range(len(...)) – explicit index  
for i in range(len(monthly_sales)):  
    print(i, monthly_sales[i])  
  
# Using enumerate() – cleaner, preferred  
for i, sales in enumerate(monthly_sales):  
    print(i, sales)
```

`enumerate()` is preferred in practice because it is more readable and less error-prone.

Accumulating Results

A common pattern is to start with an empty list and append results inside the loop.

```

above_target = []
for i, sales in enumerate(monthly_sales):
    if sales >= target:
        above_target.append(months[i])

print(above_target)    # ['Apr', 'May', 'Jun']

```

This pattern — initialise, loop, accumulate — appears constantly in data processing.

3.3 List Comprehensions

A **list comprehension** is a concise way to create a new list by applying an expression to every item in a collection. It compresses a loop into a single line.

```

# Loop version
projected = []
for sales in monthly_sales:
    projected.append(sales * 1.05)

# Comprehension version – same result
projected = [sales * 1.05 for sales in monthly_sales]

```

Filtering with a Condition

Add an `if` clause at the end to include only items that meet a condition.

```

above_target = [sales for sales in monthly_sales if sales >= 15000]

```

Inline if-else for Labelling

A different `if` placement — in the middle — produces one value or another based on a condition. This is useful for labelling.

```

labels = ["High" if sales >= 15000 else "Low" for sales in monthly_sales]

```

Note the position of `if` differs between filtering (`if` at the end) and labelling (`if` in the middle). These are two distinct patterns.

Purpose	Syntax
Filter items	<code>[x for x in data if condition]</code>
Transform items	<code>[expression for x in data]</code>
Label items	<code>[a if condition else b for x in data]</code>

3.4 Frequency Distributions

■ Stats Connection

A **frequency distribution** shows how often each value or category appears in a dataset. It is typically the first step in understanding a categorical variable.

Type	Description	Example
Frequency	Count of occurrences	Northeast: 7 transactions
Relative frequency	Proportion of total	Northeast: 35%
Cumulative frequency	Running total	Top 2 regions: 63% of transactions

Frequency distributions are most meaningful for **nominal** and **ordinal** data — region, payment method, product category, rating tier. For continuous data (revenue, temperature), you group values into bins first (covered in Week 5 with histograms).

Building a Frequency Table Manually

Python dictionaries map keys to values, making them a natural structure for frequency tables.

```
regions = ["Northeast", "West", "Northeast", "South", "West", "Northeast", ...]

freq = {}
for region in regions:
    if region in freq:
        freq[region] += 1
    else:
        freq[region] = 1
```

Using Counter

`collections.Counter` automates this in one line and adds useful methods.

```
from collections import Counter

freq = Counter(regions)
print(freq)
# Counter({'Northeast': 7, 'West': 6, 'South': 4, 'Midwest': 3})

# Most common values
freq.most_common(2)
# [('Northeast', 7), ('West', 6)]
```

Relative Frequency

Convert counts to proportions by dividing by the total.

```
total = len(regions)
for region, count in freq.items():
    pct = (count / total) * 100
    print(f"{region}: {count} ({pct:.1f}%)")
```

3.5 Outlier Detection

■ Stats Connection

An **outlier** is a value that falls unusually far from the rest of the data. In business data, outliers are common and meaningful — they can represent exceptional sales, data entry errors, fraud, or genuine anomalies worth investigating.

Identifying outliers is not about removing them. It is about knowing they exist and deciding whether they should be included in a given analysis.

The 2 Standard Deviation Rule

A simple and widely used threshold: flag any value more than two standard deviations from the mean.

Outlier if: $|x - \bar{x}| > 2\sigma$

```
mean = np.mean(transactions)
std = np.std(transactions)

outliers = [x for x in transactions if abs(x - mean) > 2 * std]
```

`abs()` computes the absolute value — the distance from the mean regardless of direction — so both unusually high and unusually low values are caught.

Limitation of This Method

The mean and standard deviation are themselves sensitive to outliers. A very large outlier inflates the standard deviation, widening the threshold and making it harder to flag the outlier in the first place. In Week 4, we introduce a more robust method based on the **interquartile range (IQR)**, which is not affected by extreme values.

The 2 standard deviation rule is a useful starting point. For production data pipelines, the IQR method is generally preferred.

3.6 Putting It Together — A Classification and Summary Workflow

A realistic data task often combines all three concepts from this chapter.

```

purchases = [320, 4800, 750, 12500, 980, 6200, 430, 8900, 1100, 275]

# Step 1 – Classify with a comprehension
tiers = ["Large" if x >= 5000 else "Medium" if x >= 1000 else "Small"
         for x in purchases]

# Step 2 – Frequency distribution
from collections import Counter
tier_freq = Counter(tiers)

# Step 3 – Relative frequency
total = len(purchases)
for tier, count in tier_freq.most_common():
    pct = (count / total) * 100
    print(f"{tier}: {count} ({pct:.1f}%)")

# Step 4 – Outlier check
import numpy as np
arr = np.array(purchases)
mean = np.mean(arr)
std = np.std(arr)
outliers = [x for x in purchases if abs(x - mean) > 2 * std]
print("Outliers:", outliers)

```

This four-step pattern — classify, count, proportion, flag — is the foundation of exploratory data analysis (EDA), which we build on in Week 5 and 6.

3.7 Chapter Summary

Concept	Key Point
if / elif / else	Evaluates conditions top-to-bottom, runs first true block
== vs =	== compares, = assigns — do not confuse them
and / or	Combine conditions; and requires both true, or requires one
for loop	Repeats for every item in a collection
enumerate()	Returns index and value together — preferred over range(len(...))
List comprehension	[expr for x in data if cond] — compact loop in one line
Frequency distribution	Count of each category — first step for nominal/ordinal data

Counter	Builds a frequency table in one line with <code>.most_common()</code>
Outlier (2 std rule)	Flag values where <code>abs(x - mean) > 2 * std</code>

Review Questions

1. What is wrong with the following code? How would you fix it?

```
if sales = 15000:
    print("On target")
```

2. A list contains 100 transaction amounts. Write a single list comprehension that returns only the transactions above \$5,000, with each value increased by 8%.
3. Explain in plain language why the 2 standard deviation outlier rule can underperform when the data contains a very large outlier.
4. You have a list of product categories from 200 orders. Write the code to build a frequency table and print each category with its count and percentage share, sorted from most to least common.
5. The Counter method `.most_common(n)` returns the top `n` items. What does `.most_common(1)[0][0]` return? Walk through each step of that expression.

Chapter 4 — Functions, Error Handling, and an Introduction to Pandas

4.1 Why Functions Matter

By Week 3, you were writing the same calculations repeatedly — mean, percentage change, outlier detection. Functions eliminate that repetition. Write the logic once, give it a name, and call it whenever you need it.

Beyond convenience, functions make code easier to read, test, and fix. When a calculation is wrong, you fix it in one place instead of hunting through every cell where you copy-pasted it.

4.2 Defining and Calling Functions

```
def function_name(parameter1, parameter2):  
    # code  
    return result
```

- `def` declares the function
- **Parameters** are the inputs the function expects
- `return` sends a value back to the caller — without it, the function returns `None`

```
def pct_change(old, new):  
    change = ((new - old) / old) * 100  
    return change
```

```
print(pct_change(48000, 54600))    # 13.75  
print(pct_change(15000, 13500))    # -10.0
```

The function is defined once and called twice with different arguments. The logic lives in one place.

Default Parameters

Parameters can have **default values**. If the caller omits that argument, the default is used.

```
def project_revenue(revenue, growth_rate=0.05):  
    return revenue * (1 + growth_rate)  
  
project_revenue(100000)            # 105000.0 — uses 5% default  
project_revenue(100000, 0.12)      # 112000.0 — uses 12%
```

Default parameters should always come after non-default ones in the function signature.

Returning Multiple Values

A function can return more than one value. Python packs them into a tuple, which can be unpacked on the receiving end.

```
def summary_stats(data):  
    return np.mean(data), np.median(data), np.std(data, ddof=1)  
  
mean, median, std = summary_stats(sales)
```

4.3 Error Handling

Functions can receive unexpected inputs — an empty list, a string where a number was expected, a zero where a denominator was assumed to be non-zero. Unhandled, these cause crashes. `try / except` lets you handle them gracefully.

```
try:  
    # code that might fail  
except ErrorType:  
    # what to do if it fails  
  
def safe_pct_change(old, new):  
    try:  
        return round(((new - old) / old) * 100, 2)  
    except ZeroDivisionError:  
        return "Error: base value cannot be zero"
```

You can handle multiple error types in sequence.

```
def safe_mean(data):  
    try:  
        return sum(data) / len(data)  
    except TypeError:  
        return "Error: data must be a list of numbers"  
    except ZeroDivisionError:  
        return "Error: list is empty"
```

Common Error Types

Error	Cause
ZeroDivisionError	Dividing by zero
TypeError	Wrong data type (e.g., adding a string to a number)
ValueError	Right type, wrong value (e.g., <code>int("abc")</code>)

IndexError	Accessing a list index that doesn't exist
KeyError	Accessing a dictionary key that doesn't exist

Error handling is not about catching every possible failure — it is about anticipating the ones that are likely given your data and use case.

4.4 Introduction to Pandas

NumPy arrays are powerful for numerical computation, but they have no column names, no row labels, and no support for mixed data types in the same structure. Real datasets have all of these.

Pandas provides two structures designed for real-world tabular data.

Series

A *Series* is a single column of data with an index. Think of it as a labelled NumPy array.

```
import pandas as pd

monthly = pd.Series([14200, 13850, 16400, 15900, 17200, 18100],
                    index=["Jan", "Feb", "Mar", "Apr", "May", "Jun"])
print(monthly["Mar"])    # 16400
```

DataFrame

A *DataFrame* is a table of rows and columns — the Pandas equivalent of a spreadsheet. Each column is a *Series*.

```
data = {
    "rep":      ["Chen", "Patel", "Okafor", "Rivera", "Thompson"],
    "region":   ["Northeast", "West", "South", "Midwest", "Northeast"],
    "q1_sales": [42100, 38900, 45300, 31800, 39200],
    "q2_sales": [44500, 41200, 43800, 35600, 42100]
}

df = pd.DataFrame(data)
```

Exploring a DataFrame

Method / Attribute	What It Returns
<code>df.shape</code>	(rows, columns) as a tuple
<code>df.dtypes</code>	Data type of each column
<code>df.head(n)</code>	First n rows (default 5)

<code>df.tail(n)</code>	Last n rows
<code>df.info()</code>	Column names, types, and non-null counts
<code>df.describe()</code>	Summary statistics for numeric columns

Accessing Data

```
# Single column – returns a Series
df["q1_sales"]

# Multiple columns – returns a DataFrame
df[["rep", "q1_sales"]]

# Row by position
df.iloc[0]          # first row

# Row by label/condition
df.loc[df["region"] == "Northeast"]
```

Adding Calculated Columns

Pandas column arithmetic is element-wise, just like NumPy.

```
df["h1_total"]      = df["q1_sales"] + df["q2_sales"]
df["q2_growth_pct"] = ((df["q2_sales"] - df["q1_sales"]) / df["q1_sales"]) * 100
```

This is one of Pandas' most useful features — adding a derived column is one line, and it operates on the whole column at once.

4.5 Variance and Standard Deviation

The mean tells you the centre. **Variance** and **standard deviation** tell you the spread — how far values typically sit from that centre.

Why Spread Matters

Two sales reps can have the same mean monthly revenue but very different reliability. Rep A consistently hits close to target. Rep B swings wildly — some months exceptional, some months poor. The mean obscures this. Spread measures reveal it.

Variance

Variance is the average squared deviation from the mean.

Sample variance (use this when working with a sample, which is almost always):

$$s^2 = \frac{\sum (x - \bar{x})^2}{n-1}$$

The $n-1$ denominator (Bessel's correction) adjusts for the fact that a sample tends to underestimate the true population spread.

Standard Deviation

Standard deviation is the square root of variance. This brings the units back to the original scale — dollars, not dollars squared.

$$s = \sqrt{s^2}$$

```
rep_a = np.array([15200, 15800, 14900, 16100, 15400, 15600])
rep_b = np.array([9800, 21000, 12500, 18900, 11200, 19600])

np.std(rep_a, ddof=1)    # ~434 — tightly clustered
np.std(rep_b, ddof=1)    # ~4,727 — highly variable
```

Both reps average approximately \$15,500. The standard deviation tells the real story.

ddof=1: NumPy's `np.std()` defaults to population standard deviation (`ddof=0`). Always pass `ddof=1` for sample data. Pandas' `.std()` defaults to `ddof=1` — the correct choice.

4.6 Z-Scores

A **z-score** expresses a value as its distance from the mean, measured in standard deviations.

$$z = \frac{x - \bar{x}}{s}$$

Z-score	Interpretation
0	Exactly at the mean
+1.5	1.5 standard deviations above the mean
-2.1	2.1 standard deviations below the mean
z	z

Uses of Z-Scores

Comparing across scales. A rep in a high-volume region and a rep in a low-volume region cannot be compared by raw sales. Z-scores normalise both to the same scale, making comparison fair.

Outlier detection. Values with $|z| > 2$ (or sometimes $|z| > 3$) are flagged for review.

Standardisation for further analysis. Some statistical models require standardised inputs.

```
z_scores = (regional_sales - np.mean(regional_sales)) / np.std(regional_sales,
ddof=1)
```

4.7 The Interquartile Range and Robust Outlier Detection

Quartiles and the IQR

Percentiles divide a sorted dataset into 100 equal parts. The key ones:

- **Q1 (25th percentile):** 25% of values fall below this point
- **Q2 (50th percentile):** the median
- **Q3 (75th percentile):** 75% of values fall below this point

The **interquartile range** is the distance between Q1 and Q3:

$$IQR = Q3 - Q1$$

The IQR covers the middle 50% of the data. It is not affected by extreme values in the tails.

The Tukey Fence Rule

The standard IQR-based outlier detection rule (Tukey fences) flags values outside:

$$\text{Lower fence} = Q1 - 1.5 \times IQR$$

$$\text{Upper fence} = Q3 + 1.5 \times IQR$$

```
q1      = np.percentile(data, 25)
q3      = np.percentile(data, 75)
iqr     = q3 - q1
lower_fence = q1 - 1.5 * iqr
upper_fence = q3 + 1.5 * iqr

outliers = data[(data < lower_fence) | (data > upper_fence)]
```

IQR vs Standard Deviation Rule

Method	Based On	Affected by Outliers?	Best Used When
2 std rule	Mean, std	Yes	Roughly symmetric data
IQR (Tukey)	Quartiles	No	Skewed data or when outliers are already suspected

In business data — where a single bulk order or executive salary can skew a dataset significantly — the IQR method is generally more reliable.

4.8 Chapter Summary

Concept	Key Point
Function	<code>def name(params): return value</code> — write once, call anywhere
Default parameter	Provides a fallback value when the argument is omitted
<code>try / except</code>	Handles anticipated errors without crashing
Pandas Series	A single labelled column of data
Pandas DataFrame	A labelled table — rows, columns, mixed types
<code>ddof=1</code>	Use for sample standard deviation in NumPy
Variance	Average squared deviation — measures spread
Standard deviation	Square root of variance — same units as data
Z-score	Deviation from mean in std units — enables comparison and outlier detection
IQR	Spread of middle 50% — not affected by extreme values
Tukey fences	$\$Q1 - 1.5 \times IQR$ and $\$Q3 + 1.5 \times IQR$

Review Questions

1. What is the difference between a **parameter** and an **argument**? Give an example of each.
2. A function `calculate_growth(revenue, rate)` is called as `calculate_growth(50000)`. What happens, and how would you fix the function definition to allow this call to work with a default rate of 3%?
3. Two datasets have the same mean of \$42,000. Dataset A has a standard deviation of \$1,200. Dataset B has a standard deviation of \$9,800. What does this tell you about each dataset?
4. A sales figure has a z-score of +2.4. What does this mean in plain language? Would you flag it as an outlier?
5. Explain in your own words why the IQR-based outlier rule is more robust than the 2 standard deviation rule when the data already contains outliers.

Chapter 5 — Data Cleaning and Visualisation

5.1 Why Data Cleaning Comes First

Raw data is rarely analysis-ready. Before drawing conclusions, you need to know what you are actually working with. Missing values, invalid entries, and inconsistent formats all produce misleading results if left unaddressed.

Data cleaning is not a preliminary chore — it is part of the analysis. Decisions made during cleaning (what to fill, what to drop, what to flag) directly shape what the data says.

5.2 Loading Data

In real work, data comes from files. The most common format is CSV (comma-separated values).

```
import pandas as pd

df = pd.read_csv("quarterly_sales.csv")
```

Once loaded, your first task is to understand what you have.

```
df.shape      # (rows, columns)
df.dtypes     # data type of each column
df.head()     # first 5 rows
df.info()     # column names, types, non-null counts
df.describe() # summary statistics for numeric columns
```

`df.describe()` returns count, mean, std, min, quartiles, and max for every numeric column. It is often the first statistical summary you run on a new dataset.

5.3 Missing Data

Identifying Missing Values

```
df.isnull().sum()          # count per column
df.isnull().sum() / len(df) * 100  # as percentage
```

Strategies for Handling Missing Data

Strategy	When to Use
----------	-------------

Drop rows	Few missing values, missing at random
Fill with median	Numeric data, distribution may be skewed
Fill with mean	Numeric data, distribution is symmetric
Fill with mode	Categorical data
Fill with a constant	When a specific value (e.g., 0) makes business sense
Flag and investigate	When the missingness itself may be meaningful

```
# Fill missing revenue with the median
df["revenue"] = df["revenue"].fillna(df["revenue"].median())

# Drop rows where any value is missing
df.dropna(inplace=True)
```

Why median over mean for imputation? If the distribution is skewed — for example, a few very large transactions in a sales dataset — the mean is pulled toward the high end. Filling missing values with the mean implicitly assumes the missing observations were above-average, which may not be true. The median is more robust.

Why not fill with zero? Zero means something: no revenue, no units sold. If a transaction occurred but its value is unknown, filling with zero misrepresents it as a transaction with zero value. This is a common mistake that quietly distorts analysis.

5.4 Data Quality

Missing values are only one category of data problem. Values can also be present but wrong.

Common Quality Issues in Business Data

Issue	Example
Negative values in non-negative fields	revenue = -450
Out-of-range values	discount_pct = 1.50 (150% discount)
Impossible combinations	units_sold = 0 but revenue = 12000
Duplicate rows	Same transaction recorded twice

Finding and Fixing Invalid Values

```
# Find rows with invalid discounts
invalid = df[df["discount_pct"] < 0]

# Replace with 0
df.loc[df["discount_pct"] < 0, "discount_pct"] = 0

# Check for duplicates
df.duplicated().sum()

# Remove duplicates
df.drop_duplicates(inplace=True)
```

Safe Division with `np.where`

When adding a calculated column that involves division, guard against zero denominators.

```
df["revenue_per_unit"] = np.where(
    df["units_sold"] > 0,
    df["revenue"] / df["units_sold"],
    np.nan
)
```

`np.where(condition, value_if_true, value_if_false)` is a vectorised if-else — it operates on the entire column at once.

5.5 Visualisation with Matplotlib and Seaborn

Python has two main visualisation libraries:

Library	Role
Matplotlib	Low-level, full control, verbose syntax
Seaborn	Built on Matplotlib, simpler syntax for statistical plots

Most analysts use both — Seaborn for quick, clean statistical charts; Matplotlib for fine-grained customisation.

```
import matplotlib.pyplot as plt
import seaborn as sns

sns.set_theme(style="whitegrid")  # clean default style
```

5.6 Histograms

A **histogram** bins a continuous variable and plots frequency per bin. It reveals the **shape** of the distribution.

```
# Matplotlib
plt.hist(df["revenue"], bins=20, color="steelblue", edgecolor="white")
plt.axvline(df["revenue"].mean(), color="red", linestyle="--", label="Mean")
plt.axvline(df["revenue"].median(), color="orange", linestyle="--", label="Median")
plt.legend()
plt.show()

# Seaborn with KDE overlay
sns.histplot(df["revenue"], bins=20, kde=True)
plt.show()
```

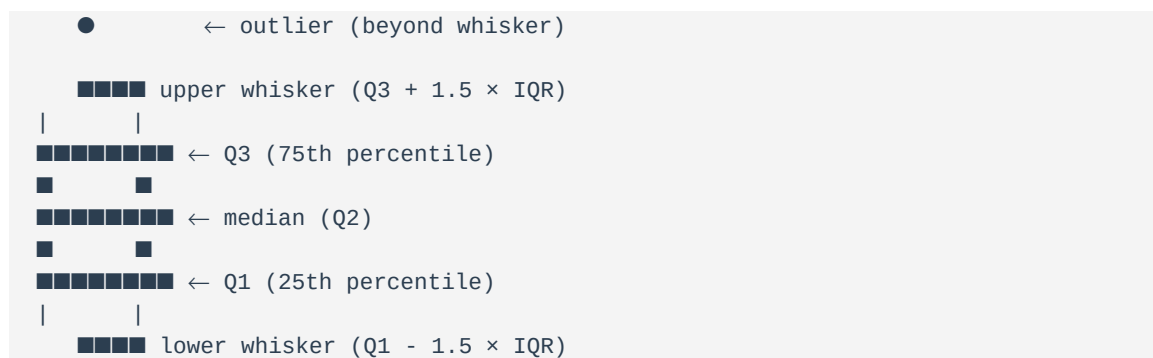
Distribution Shapes

Shape	Description	Mean vs Median
Symmetric	Mirror image left and right	Mean $\approx$ Median
Right-skewed	Long tail to the right	Mean $>$ Median
Left-skewed	Long tail to the left	Mean $<$ Median
Bimodal	Two peaks	Two sub-populations likely present

The KDE (kernel density estimate) is a smoothed curve that makes the shape easier to read than raw bars, especially with smaller datasets.

5.7 Boxplots

A **boxplot** (box-and-whisker plot) visualises the five-number summary and highlights outliers.



Boxplots are especially useful for **comparing groups** — plotting one box per category side by side immediately shows differences in centre, spread, and outliers.

```
sns.boxplot(data=df, x="region", y="revenue", palette="Set2", hue="region",
            legend=False)
plt.title("Revenue by Region")
plt.show()
```


5.8 Scatter Plots

A **scatter plot** shows the relationship between two continuous variables. Each row in the DataFrame becomes one point.

```
sns.scatterplot(data=df, x="units_sold", y="revenue",
                hue="region", alpha=0.7)
plt.title("Units Sold vs Revenue")
plt.show()
```

The `hue` parameter colours points by a categorical variable, adding a third dimension to the chart.

What to Look For

Pattern	What It Suggests
Points rising left to right	Positive relationship
Points falling left to right	Negative relationship
No clear direction	Weak or no linear relationship
Curved pattern	Non-linear relationship
Tight clustering	Strong relationship
Diffuse cloud	Weak relationship

5.9 Correlation

Correlation quantifies the strength and direction of the linear relationship between two continuous variables.

The **Pearson correlation coefficient** r :

$$r = \frac{\sum (x - \bar{x})(y - \bar{y})}{(n-1) \cdot s_x \cdot s_y}$$

r ranges from -1 to $+1$:

r value	Interpretation
$+1$	Perfect positive linear relationship
$+0.7$ to $+1$	Strong positive

\$+0.4\$ to \$+0.7\$	Moderate positive
\$-0.4\$ to \$+0.4\$	Weak or none
\$-0.7\$ to \$-0.4\$	Moderate negative
\$-1\$ to \$-0.7\$	Strong negative
\$-1\$	Perfect negative linear relationship

```
# Single correlation
df["units_sold"].corr(df["revenue"])

# Full correlation matrix
df[["revenue", "units_sold", "discount_pct"]].corr()
```

Visualising the Correlation Matrix

```
corr = df[["revenue", "units_sold", "discount_pct"]].corr()

sns.heatmap(corr, annot=True, cmap="coolwarm", center=0, fmt=".2f")
plt.title("Correlation Heatmap")
plt.show()
```

The heatmap uses colour to communicate direction (warm = positive, cool = negative) and annotation to show the exact value.

Correlation Is Not Causation

A high correlation between two variables does not mean one causes the other. Both could be driven by a third variable, or the relationship could be coincidental. Correlation is a starting point for investigation, not a conclusion.

5.10 Subplots

When you want multiple charts together, use `plt.subplots()`.

```
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

# First chart
axes[0].hist(df["revenue"], bins=20)
axes[0].set_title("Revenue")

# Second chart
axes[1].hist(df["units_sold"], bins=15)
axes[1].set_title("Units Sold")

plt.tight_layout()
plt.show()
```

`tight_layout()` prevents overlapping labels. For a 2×2 grid, index axes as `axes[row, col]`.

5.11 Chapter Summary

Concept	Key Point
<code>df.info()</code> / <code>df.describe()</code>	First steps when loading new data
<code>isnull().sum()</code>	Count missing values per column
<code>fillna(median)</code>	Preferred for skewed numeric columns
Do not fill with zero	Zero means something — misrepresents unknowns
<code>df.loc[cond, col] = value</code>	Fix invalid values in place
<code>np.where()</code>	Vectorised if-else for calculated columns
Histogram	Shows distribution shape and skew
KDE	Smoothed distribution curve
Boxplot	Five-number summary — good for comparing groups
Scatter plot	Visualises relationship between two continuous variables
Pearson r	Linear correlation, -1 to $+1$
Heatmap	Colour-coded correlation matrix
<code>plt.subplots()</code>	Multiple charts in one figure

Review Questions

1. A column `price` has 12 missing values out of 200 rows. The distribution of prices is right-skewed due to a few luxury items. Would you fill with mean or median? Justify your answer.
2. What is the difference between a missing value and an invalid value? Give one example of each from a business sales dataset.
3. A histogram shows the revenue distribution has a long right tail. Based on this, would you expect the mean to be higher or lower than the median? Explain.

4. Two boxplots show revenue by region. Region A has a tall box; Region B has a short box. What does this tell you about the sales teams in each region?
 5. A scatter plot of advertising spend (x) and revenue (y) shows a strong positive correlation of $r = 0.82$. A colleague concludes that increasing advertising spend causes revenue to increase. What caution would you offer about this interpretation?
-

Chapter 6 — Grouped Analysis, Pivot Tables, and the EDA Workflow

6.1 From Individual Rows to Group Summaries

The analyses in previous weeks operated on the dataset as a whole — overall mean, overall distribution, overall correlation. Most business questions are not about the whole; they are about segments: *Which region performs best? Does product mix vary by quarter? Which rep is most consistent?*

Answering these questions requires splitting the data into groups and summarising each group separately. Pandas makes this straightforward with `groupby`.

6.2 Grouped Analysis with `groupby`

`groupby` follows a **split-apply-combine** pattern:

1. **Split** the DataFrame into groups based on a categorical column
2. **Apply** an aggregation function to each group
3. **Combine** the results into a new DataFrame

```
df.groupby("region")["revenue"].mean()
```

This reads as: *group the rows by region, then calculate the mean revenue for each group.*

Applying Multiple Statistics — `.agg()`

`.agg()` applies several functions at once and lets you name the output columns.

```
df.groupby("region")["revenue"].agg(  
    count = "count",  
    total = "sum",  
    mean = "mean",  
    median = "median",  
    std = "std"  
)
```

Grouping by Multiple Columns

Pass a list to `groupby` to create groups defined by combinations of categories.

```
df.groupby(["rep", "quarter"])["revenue"].mean()
```

This returns the mean revenue for every rep-quarter combination — a natural way to track performance over time by individual.

6.3 The Coefficient of Variation

When comparing spread across groups with different means, absolute standard deviation is misleading. A group with a higher mean will naturally tend to have a higher standard deviation in the same units, even if it is proportionally no more variable.

The **coefficient of variation (CV)** expresses standard deviation as a percentage of the mean, making it scale-independent.

$$CV = \frac{s}{\bar{x}} \times 100$$

```
region_stats = df.groupby("region")["revenue"].agg(mean="mean", std="std")
region_stats["cv_pct"] = (region_stats["std"] / region_stats["mean"] * 100).round(1)
```

A region with a mean of \$50,000 and std of \$8,000 has CV = 16%. A region with mean \$30,000 and std of \$6,000 has CV = 20%. Despite the lower absolute std, the second region is more variable relative to its own baseline.

CV is useful for comparing consistency across reps, regions, or time periods — especially when their revenue scales differ.

6.4 Pivot Tables

A **pivot table** cross-tabulates two categorical variables and fills each cell with an aggregated value.

```
df.pivot_table(
    values = "revenue",
    index  = "region",      # row categories
    columns = "quarter",    # column categories
    aggfunc = "mean"        # aggregation function
)
```

The result is a matrix where each cell contains the mean revenue for a specific region-quarter combination. This format makes it easy to scan for patterns — which region-quarter had the highest mean, where there might be seasonal effects, which combinations are weak.

Common Aggregation Functions

aggfunc	What It Computes
---------	------------------

"mean"	Average value per cell
"sum"	Total value per cell
"count"	Number of observations per cell
"median"	Median value per cell

Adding Row and Column Totals

Pass `margins=True` to include row and column totals automatically.

```
df.pivot_table(values="revenue", index="region",
               columns="quarter", aggfunc="sum", margins=True)
```

Visualising a Pivot Table

A pivot table is a matrix — a heatmap is its natural visual representation.

```
pivot = df.pivot_table(values="revenue", index="region",
                      columns="quarter", aggfunc="mean").round(0)

sns.heatmap(pivot, annot=True, fmt=".0f", cmap="YlGnBu", linewidths=0.5)
plt.title("Mean Revenue by Region and Quarter")
plt.show()
```

The colour gradient lets the eye quickly find the highest and lowest cells, while the annotations give the exact values.

6.5 The EDA Workflow

Exploratory data analysis (EDA) is a systematic process for understanding a new dataset before drawing conclusions or building models. It combines everything from this course into a structured sequence.

The Five-Step EDA Workflow

Step	Goal	Key Questions
1. Structure	Understand what you have	How many rows and columns? What are the data types?
2. Data quality	Find and fix problems	Missing values? Invalid entries? Duplicates?

3. Univariate analysis	Understand each variable	What is the distribution shape? Mean vs median? Any outliers?
4. Bivariate analysis	Find relationships	How do pairs of variables relate? What correlates with the outcome?
5. Grouped analysis	Find segment patterns	Do relationships or distributions differ across categories?

Step 1 — Structure

```
print(df.shape)
print(df.dtypes)
df.head()
df.describe()
```

Step 2 — Data Quality

```
df.isnull().sum()
(df["discount_pct"] < 0).sum()
df.duplicated().sum()
```

Always fix quality issues before any analysis. Findings built on dirty data are unreliable.

Step 3 — Univariate Analysis

For each variable:

- **Numeric:** histogram, mean, median, std, IQR, outlier check
- **Categorical:** frequency table, bar chart, most/least common

The histogram and the mean-vs-median comparison answer the same question from two angles — is the distribution symmetric or skewed? Are they consistent with each other?

Step 4 — Bivariate Analysis

- Scatter plots for pairs of numeric variables
- Correlation matrix and heatmap
- Boxplots of a numeric variable split by a categorical variable

Step 5 — Grouped Analysis

- groupby with .agg() for mean, std, CV by category
- Pivot tables for two-way breakdowns
- Visualise with boxplots and heatmaps

6.6 From Analysis to Narrative

Numbers are not findings. A finding is a number plus an interpretation:

Just a number	A finding
Northeast mean = \$46,200	The Northeast region leads all regions in mean revenue, outperforming the next closest (West at \$43,800) by approximately 5%.
Correlation = 0.03	Discount level has no meaningful linear relationship with revenue in this dataset.
CV (South) = 22%	The South region is the most variable — nearly twice the CV of the Midwest (12%) — suggesting inconsistent performance that may warrant investigation.

Business analysts are expected to translate numbers into sentences that a non-technical stakeholder can act on. The statistics provide evidence; the narrative provides meaning.

6.7 Choosing the Right Summary

A quick reference for the decision points covered across all six weeks:

Question	Tool
What is the centre of the data?	Mean (symmetric data), Median (skewed or outliers present)
How spread out is the data?	Std deviation, IQR
How does spread compare across groups with different scales?	CV
Is a value unusually far from the rest?	Z-score, IQR fences
How often does each category appear?	Frequency table, Counter
How does a variable distribute across its range?	Histogram
How do groups compare on a numeric variable?	Boxplot
Is there a relationship between two numeric variables?	Scatter plot, Pearson r

How does a numeric variable break down across two categories?	Pivot table, heatmap
---	----------------------

6.8 Chapter Summary

Concept	Key Point
groupby	Split by category, apply a function, combine results
.agg()	Multiple statistics in one call with named output columns
CV	$s / \bar{x} \times 100$ — relative spread, comparable across groups
Pivot table	Two-way cross-tabulation — rows, columns, aggregated values
margins=True	Adds row and column totals to a pivot table
Heatmap of pivot	Natural visual for a cross-tabulated matrix
EDA workflow	Structure → quality → univariate → bivariate → grouped
Analysis narrative	Interpret numbers in plain language — what does it mean and what should be done?

Review Questions

1. What does the groupby "split-apply-combine" pattern mean? Describe each step in plain language.
2. Two sales regions have the following statistics. Which is more consistent, and why?
 - Region A: mean = \$52,000, std = \$9,100
 - Region B: mean = \$38,000, std = \$7,200
3. You create a pivot table showing total revenue by product (rows) and quarter (columns). One cell shows a very high value compared to the others. What are two possible explanations, and how would you investigate further?
4. Write the Pandas code to create a summary table showing mean, median, and standard deviation of `units_sold` grouped by `rep`, with the results sorted by mean in descending order.

5. You present your EDA findings to the sales director. She points to your highest-performing region and says: "Let's just focus all our resources there." What statistical information from your analysis would you use to give a more nuanced recommendation?

This is the final chapter. You have completed Introduction to Python for Business Statistics.

What Comes Next

This course built a foundation. Where it leads depends on your goals:

Direction	Next Steps
More statistics	Hypothesis testing, regression, probability distributions
More Python	Object-oriented programming, APIs, automation
Data engineering	SQL, databases, data pipelines
Machine learning	scikit-learn, feature engineering, model evaluation
Visualisation	Advanced Seaborn, Plotly for interactive charts, dashboards

The EDA workflow from this course is the starting point for all of them.

License

Creative Commons Attribution 4.0 International (CC BY 4.0)

© 2026 Patrick Dolinger

This work is licensed under the Creative Commons Attribution 4.0 International License.

You are free to:

- **Share** — copy and redistribute the material in any medium or format
- **Adapt** — remix, transform, and build upon the material for any purpose

Under the following terms:

- **Attribution** — You must give appropriate credit to the author, provide a link to the license, and indicate if changes were made.

[Creative Commons Attribution 4.0 International \(CC BY 4.0\)](#)